TECHNICAL TRAINING COURSE

Java API for RESTful Services

JAX-RS 2.x | javax.ws.rs | XML over HTTP

Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1 (Jersey 2.x)
Namespace: javax.ws.rs.* (NOT jakarta.ws.rs.*)
Data format: XML (MediaType.APPLICATION_XML / text/xml)
Duration: 1 Day (8 hours)
Level: Intermediate — Java Developer

Table of Contents

1. Applications

1.1 Learning Objectives

- Understand what a JAX-RS Application is and its role in the framework
- Create a JAX-RS application class with @ApplicationPath
- Understand component registration: automatic scanning vs manual registration
- Configure a JAX-RS application inside Tomcat 9 with and without web.xml

1.2 What is a JAX-RS Application?

A JAX-RS Application is the entry point to the framework. The Application class (javax.ws.rs.core.Application) serves two purposes: it declares the base URI path under which all resources are rooted, and it optionally provides an explicit set of resource and provider classes to register. Every JAX-RS deployment must have exactly one Application subclass.

Term	Definition
Application	Abstract base class in javax.ws.rs.core that defines the JAX-RS component model.
@ApplicationPath	Annotation on an Application subclass that sets the base URI path for all resources.
Resource class	A POJO annotated with @Path; handles HTTP requests for one or more URIs.
Provider	A class annotated with @Provider; extends the JAX-RS runtime (serialisation, filters, etc.).

1.3 Minimal Application Class

The simplest possible JAX-RS application requires only two annotations. The code in this course uses the javax.ws.rs.* package, which is correct for Tomcat 9:

> **Package Namespace — All code uses javax.ws.rs.* (NOT jakarta.ws.rs.*)**
> This module targets Java 11 / Tomcat 9.0, which use the javax.ws.rs.* package namespace.
> import javax.ws.rs.GET; import javax.ws.rs.Path; import javax.ws.rs.Produces;
> import javax.ws.rs.core.Response; import javax.ws.rs.core.MediaType;
> Do NOT use jakarta.ws.rs.* — that is for Tomcat 10 / Jakarta EE 9+.

```
package com.example.api;

import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

// All resources will be rooted at /api/*
@ApplicationPath("/api")
public class ShopApplication extends Application {
    // Empty body — JAX-RS runtime scans the classpath
    // for all @Path and @Provider classes
```

```
}
```

1.4 Manual Component Registration

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;
import java.util.Set;

@ApplicationPath("/api")
public class ShopApplication extends Application {

    // getClasses() — JAX-RS creates a NEW instance per request
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(
            ProductResource.class,
            OrderResource.class,
            // Providers
            XmlValidationExceptionMapper.class,
            NotFoundExceptionMapper.class
        );
    }

    // getSingletons() — ONE shared instance for the application lifetime
    @Override
    public Set<Object> getSingletons() {
        // Register a JAXB ContextResolver to customise marshalling
        return Set.of(new JaxbContextResolver());
    }
}
```

getClasses() vs getSingletons()

getClasses(): Returns Class<?> objects. A fresh instance is created per request.
Resource classes always go here — they are typically stateless.
getSingletons(): Returns pre-built Object instances shared across all requests.
Use for providers wrapping expensive infrastructure (JAXBContext, etc.).

If EITHER method returns a non-empty set, automatic scanning is DISABLED.
You must register EVERY class and provider manually.

1.5 Deployment in Tomcat 9

Tomcat 9 does not include a JAX-RS runtime by default. You must bundle Jersey 2.x in your WAR. For XML-producing services, the JAXB dependencies must also be declared — JAXB was removed from the JDK in Java 9:

```
<!-- pom.xml — Jersey 2.x with JAXB for XML support -->
<dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-servlet</artifactId>
    <version>2.41</version>
</dependency>
<dependency>
    <groupId>org.glassfish.jersey.inject</groupId>
```

```
    <artifactId>jersey-hk2</artifactId>
    <version>2.41</version>
</dependency>
<!-- Jersey JAXB support — enables @Produces(APPLICATION_XML) -->
<dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-jaxb</artifactId>
    <version>2.41</version>
</dependency>
<!-- JAXB API — removed from JDK in Java 9 -->
<dependency>
    <groupId>javax.xml.bind</groupId>
    <artifactId>jaxb-api</artifactId>
    <version>2.3.1</version>
</dependency>
<!-- JAXB runtime -->
<dependency>
    <groupId>com.sun.xml.bind</groupId>
    <artifactId>jaxb-impl</artifactId>
    <version>2.3.9</version>
    <scope>runtime</scope>
</dependency>
<!-- Servlet API — provided by Tomcat 9 -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
</dependency>
```

web.xml alternative

With Jersey bundled and @ApplicationPath present, no web.xml is needed. Jersey's ServletContainerInitializer auto-registers itself via the Servlet 3 SPI.

If you prefer explicit registration in web.xml, use the init-param javax.ws.rs.Application (not jakarta.ws.rs.Application) for Tomcat 9.

1.6 Review Questions

1. What happens to automatic classpath scanning when getClasses() returns a non-empty Set?
2. Why are resource classes registered with getClasses() rather than getSingletons()?
3. You deploy two JAX-RS applications in the same WAR, one at /api and one at /admin. Is this supported? What must be unique?

2. Resources

2.1 Learning Objectives

- Create JAX-RS root resource classes and methods using core annotations
- Use @Path, HTTP method annotations, @Produces(APPLICATION_XML), and @Consumes correctly
- Inject request parameters using @PathParam, @QueryParam, @HeaderParam
- Build and return Response objects with correct HTTP status codes and headers
- Understand how JAXB annotated classes are automatically serialised to XML

2.2 Root Resource Classes — XML Producing

A root resource class annotated with @Produces(MediaType.APPLICATION_XML) causes JAX-RS to serialise return values using JAXB. The domain model classes must carry @XmlRootElement for this to work:

```
package com.example.api.resource;

import javax.ws.rs.*;
import javax.ws.rs.core.*;
import com.example.model.Product;
import java.util.List;

@Path("/products")
@Produces(MediaType.APPLICATION_XML)    // all methods return XML by default
@Consumes(MediaType.APPLICATION_XML)    // all methods accept XML bodies
public class ProductResource {

    private final ProductRepository repo = ProductRepository.getInstance();

    // GET /api/products  →  <products><product>...</product></products>
    @GET
    public List<Product> listAll() {
        return repo.findAll();
    }

    // GET /api/products/P001  →  <product><id>P001</id>...</product>
    @GET
    @Path("{id}")
    public Response getById(@PathParam("id") String id) {
        Product p = repo.findById(id);
        if (p == null) {
            return Response.status(Response.Status.NOT_FOUND).build();
        }
        return Response.ok(p).build();      // JAXB marshals Product → XML
    }

    // POST /api/products  body: <product>...</product>
    @POST
    public Response create(Product product, @Context UriInfo uriInfo) {
        Product saved = repo.save(product);
        URI location  = uriInfo.getAbsolutePathBuilder()
                               .path(saved.getId()).build();
        return Response.created(location)   // 201 Created + Location header
                       .entity(saved)       // JAXB marshals saved product
                       .build();
```

```
    }

    // PUT /api/products/P001  body: <product>...</product>
    @PUT
    @Path("{id}")
    public Response update(@PathParam("id") String id,
                           Product product) {
        Product updated = repo.update(id, product);
        if (updated == null)
            return Response.status(Response.Status.NOT_FOUND).build();
        return Response.ok(updated).build();
    }

    // DELETE /api/products/P001  →  204 No Content
    @DELETE
    @Path("{id}")
    public void delete(@PathParam("id") String id) {
        repo.delete(id);
    }
}
```

2.3 JAXB-Annotated Domain Model

For JAX-RS to automatically serialise a class to XML, it must carry @XmlRootElement. Jersey uses the JAXB runtime (jaxb-impl) to marshal the return value:

```
package com.example.model;

import javax.xml.bind.annotation.*;
import java.math.BigDecimal;

@XmlRootElement(name = "product")
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = {
    "id", "name", "category", "price", "stockLevel", "active"
})
public class Product {

    @XmlAttribute
    private String     id;
    @XmlElement(required = true)
    private String     name;
    @XmlElement(required = true)
    private String     category;
    @XmlElement(required = true)
    private BigDecimal price;
    @XmlElement
    private int        stockLevel;
    @XmlElement
    private boolean    active;

    public Product() {}   // no-arg constructor required by JAXB
    // ... getters and setters
}

// The marshalled XML for one product:
// <product id="P001">
//   <name>Clean Code</name>
//   <category>books</category>
//   <price>39.99</price>
//   <stockLevel>50</stockLevel>
```

```
//    <active>true</active>
// </product>
```

XML Collection Wrapper

When a resource method returns List<Product>, Jersey wraps the items.
To control the wrapper element name, use @XmlRootElement on a wrapper class:

```
@XmlRootElement(name = "products")
public class ProductList {
   @XmlElement(name = "product")
   private List<Product> items;
}
```

Or annotate the list field in the enclosing class with @XmlElementWrapper.

2.4 Core Resource Annotations

Term	Definition
@Path	URI path template for a class or method. Supports {variable} templates and regex {id:\\d+}.
@GET	HTTP GET — safe and idempotent. Returns XML body (with @Produces APPLICATION_XML).
@POST	HTTP POST — creates a resource. Accepts XML body (with @Consumes APPLICATION_XML).
@PUT	HTTP PUT — full replacement. Must be idempotent.
@DELETE	HTTP DELETE — removes a resource. Returns 204 No Content.
@Produces	Declares Content-Type of the response. Use MediaType.APPLICATION_XML for XML.
@Consumes	Declares accepted Content-Type of the request body. Use APPLICATION_XML for XML input.
@PathParam	Binds a URI template variable: @Path("{id}") + @PathParam("id").
@QueryParam	Binds a URI query parameter (?key=value). Null if absent; combine with @DefaultValue.
@HeaderParam	Binds an HTTP request header value to a method parameter.
@Context	Injects JAX-RS context objects: UriInfo, HttpHeaders, Request, SecurityContext.
@DefaultValue	Sets a default String when the parameter is absent. Works with any @Param annotation.

2.5 The Response Builder

```
import javax.ws.rs.core.Response;
import javax.ws.rs.core.MediaType;

// 200 OK with JAXB-marshalled entity
return Response.ok(product).build();

// 201 Created with Location header and XML body
URI loc = uriInfo.getAbsolutePathBuilder().path(id).build();
return Response.created(loc).entity(saved).build();

// 204 No Content
return Response.noContent().build();

// 400 Bad Request with XML error detail
return Response.status(Response.Status.BAD_REQUEST)
               .entity(new ErrorDetail("Invalid SKU format"))
               .type(MediaType.APPLICATION_XML)
               .build();

// 404 Not Found
return Response.status(Response.Status.NOT_FOUND).build();

// Custom header
return Response.ok(product)
               .header("X-Rate-Limit-Remaining", 99)
               .build();

// Error detail bean — must be @XmlRootElement annotated
// @XmlRootElement(name = "error")
// public class ErrorDetail {
//     @XmlElement public String message;
//     @XmlElement public int    status;
// }
```

2.6 Content Negotiation — XML and plain text

A resource method can produce multiple media types. JAX-RS selects the most appropriate one by matching the client's Accept header. This allows an endpoint to serve both XML and plain text:

```
@GET
@Path("{id}/summary")
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public Response getSummary(@PathParam("id") String id,
                           @Context Request request) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();

    List<Variant> variants = Variant.mediaTypes(
        MediaType.APPLICATION_XML_TYPE,
        MediaType.TEXT_PLAIN_TYPE).build();
    Variant chosen = request.selectVariant(variants);

    if (MediaType.TEXT_PLAIN_TYPE.equals(chosen.getMediaType())) {
        return Response.ok(
            p.getId() + " | " + p.getName() + " | " + p.getPrice(),
            MediaType.TEXT_PLAIN).build();
    }
    return Response.ok(p).build();   // JAXB → XML
}
```

```
// curl -H 'Accept: application/xml'  .../products/P001/summary
// → <product id="P001"><name>Clean Code</name>...</product>

// curl -H 'Accept: text/plain'  .../products/P001/summary
// → P001 | Clean Code | 39.99

// curl -H 'Accept: text/html'  .../products/P001/summary
// → 406 Not Acceptable
```

2.7 Review Questions

4. What HTTP status code does a void resource method return? Can this be changed?
5. What JAXB annotation is required on a domain class for JAX-RS to automatically marshal it to XML?
6. Write a resource method that accepts an XML body and returns 201 Created with a Location header.
7. What happens when the client sends Accept: application/json but the resource only @Produces(APPLICATION_XML)?

3. Sub-resources

3.1 Learning Objectives

- Understand the difference between root resources and sub-resources
- Implement sub-resource methods using @Path on resource methods
- Implement sub-resource locators that return delegate objects
- Design a hierarchical URI structure using sub-resources

Term	Definition
Sub-resource method	A method with both an HTTP verb annotation AND a @Path. Handles a specific URI under the parent resource's path.
Sub-resource locator	A method with @Path but NO HTTP verb annotation. Returns an object that handles the rest of the request.

3.2 Sub-resource Methods

```
@Path("/orders")
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public class OrderResource {

    // Root: GET /api/orders  →  <orders><order>...</order></orders>
    @GET
    public List<Order> listAll() { ... }

    // Root: GET /api/orders/ORD-001  →  <order>...</order>
    @GET @Path("{id}")
    public Response get(@PathParam("id") String id) { ... }

    // Sub-resource: GET /api/orders/ORD-001/status  →  plain text
    @GET
    @Path("{id}/status")
    @Produces(MediaType.TEXT_PLAIN)
    public String getStatus(@PathParam("id") String id) {
        return service.findById(id).getStatus().name();
    }

    // Sub-resource: PUT /api/orders/ORD-001/cancel  →  204
    @PUT
    @Path("{id}/cancel")
    public Response cancel(@PathParam("id") String id) {
        service.cancel(id);
        return Response.noContent().build();
    }

    // Sub-resource: GET /api/orders/ORD-001/lines  →  XML list
    @GET
    @Path("{id}/lines")
    public List<OrderLine> getLines(@PathParam("id") String id) {
        return service.findById(id).getLines();
    }

    // Sub-resource: POST /api/orders/ORD-001/lines  body: <orderLine>...
```

```
    @POST
    @Path("{id}/lines")
    public Response addLine(@PathParam("id") String id,
                            OrderLine line,
                            @Context UriInfo ui) {
        OrderLine saved = service.addLine(id, line);
        URI loc =
ui.getAbsolutePathBuilder().path(saved.getLineId()).build();
        return Response.created(loc).entity(saved).build();
    }
}
```

3.3 Sub-resource Locators

```
@Path("/stores")
@Produces(MediaType.APPLICATION_XML)
public class StoreResource {

    // GET /api/stores  →  <stores><store>...</store></stores>
    @GET
    public List<Store> listStores() { ... }

    // Sub-resource LOCATOR — no HTTP verb annotation
    // Handles /api/stores/{storeId}/inventory/**
    @Path("{storeId}/inventory")
    public InventoryResource getInventoryResource(
            @PathParam("storeId") String storeId) {
        Store store = storeService.find(storeId);
        if (store == null)
            throw new javax.ws.rs.NotFoundException("Store: " + storeId);
        return new InventoryResource(store);
    }
}

// Sub-resource class — no @Path at class level
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public class InventoryResource {
    private final Store store;
    public InventoryResource(Store store) { this.store = store; }

    // GET /api/stores/{storeId}/inventory  →
<inventoryItems>...</inventoryItems>
    @GET
    public List<InventoryItem> getAll() { return store.getInventory(); }

    // GET /api/stores/{storeId}/inventory/{sku}  →
<inventoryItem>...</inventoryItem>
    @GET @Path("{sku}")
    public InventoryItem getBySku(@PathParam("sku") String sku) { ... }

    // PUT /api/stores/{storeId}/inventory/{sku}  body: <inventoryItem>...
    @PUT @Path("{sku}")
    public InventoryItem update(@PathParam("sku") String sku,
                                InventoryItem item) { ... }
}
```

3.4 Hierarchical URI Design

HTTP Method	URI	Operation
GET	/api/stores	List all stores
GET	/api/stores/{id}	Get a store
GET	/api/stores/{id}/inventory	Get store inventory (XML)
GET	/api/stores/{id}/inventory/{sku}	Get one inventory item
PUT	/api/stores/{id}/inventory/{sku}	Update stock level
GET	/api/orders/{id}/lines	Get order lines
POST	/api/orders/{id}/lines	Add a line item (XML body)

3.5 Review Questions

8. What is the key difference between a sub-resource method and a sub-resource locator?
9. Why is CDI injection not available in sub-resource instances created with 'new', and how do you fix this?
10. Design a URI hierarchy for a hospital system with patients, appointments, and prescriptions. Identify which would be sub-resource methods and which would use locators.

4. Providers

4.1 Learning Objectives

- Understand the JAX-RS Provider extension model
- Implement MessageBodyReader and MessageBodyWriter for custom serialisation
- Implement ExceptionMapper to convert exceptions to XML HTTP responses
- Implement ContainerRequestFilter and ContainerResponseFilter
- Control provider priority with @Priority

4.2 Provider Types

Provider Interface	Responsibility	Typical Use Cases
MessageBodyReader<T>	Deserialise request body → Java type	XML parsing, CSV parsing
MessageBodyWriter<T>	Serialise Java type → response body	Custom XML, CSV, binary formats
ExceptionMapper<E>	Convert exception → HTTP Response	NotFoundException, validation errors
ContainerRequestFilter	Intercept and modify incoming requests	Authentication, logging, rate limiting
ContainerResponseFilter	Intercept and modify outgoing responses	CORS headers, response logging
ContextResolver<T>	Supply context objects to other providers	Custom JAXBContext configuration

4.3 MessageBodyWriter — Custom XML Serialiser

When the built-in JAXB provider does not meet your needs — for example when writing a custom XML format, or serialising a type without @XmlRootElement — implement MessageBodyWriter directly:

```
import javax.ws.rs.Produces;
import javax.ws.rs.WebApplicationException;
import javax.ws.rs.core.*;
import javax.ws.rs.ext.MessageBodyWriter;
import javax.ws.rs.ext.Provider;

@Provider
@Produces("text/csv")
public class ProductCsvWriter
        implements MessageBodyWriter<List<Product>> {

    @Override
    public boolean isWriteable(Class<?> type, Type genericType,
                               Annotation[] annotations,
                               MediaType mediaType) {
        if (!List.class.isAssignableFrom(type)) return false;
        if (genericType instanceof ParameterizedType pt) {
            return Product.class.equals(
```

```
                pt.getActualTypeArguments()[0]);
        }
        return false;
    }

    @Override
    public void writeTo(List<Product> products,
                        Class<?> type, Type genericType,
                        Annotation[] annotations, MediaType mediaType,
                        MultivaluedMap<String,Object> headers,
                        OutputStream out) throws IOException {
        PrintWriter pw = new PrintWriter(
            new OutputStreamWriter(out, StandardCharsets.UTF_8));
        pw.println("id,name,category,price,stockLevel");
        products.forEach(p -> pw.printf("%s,%s,%s,%.2f,%d%n",
            p.getId(), p.getName(), p.getCategory(),
            p.getPrice(), p.getStockLevel()));
        pw.flush();
    }

    @Override
    public long getSize(List<Product> p, Class<?> t, Type g,
                        Annotation[] a, MediaType m) { return -1; }
}
```

4.4 ExceptionMapper — XML Error Responses

ExceptionMapper converts a thrown exception into an HTTP Response. For an XML API the error body should also be XML — use a JAXB-annotated error bean as the entity:

```
// Error bean — must be @XmlRootElement annotated
@XmlRootElement(name = "error")
@XmlAccessorType(XmlAccessType.FIELD)
public class ApiError {
    @XmlElement public int    status;
    @XmlElement public String code;
    @XmlElement public String message;
    public ApiError() {}
    public ApiError(int status, String code, String message) {
        this.status = status; this.code = code; this.message = message;
    }
}

// ExceptionMapper — returns XML error body
import javax.ws.rs.NotFoundException;
import javax.ws.rs.core.Response;
import javax.ws.rs.ext.ExceptionMapper;
import javax.ws.rs.ext.Provider;

@Provider
public class NotFoundExceptionMapper
        implements ExceptionMapper<NotFoundException> {
    @Override
    public Response toResponse(NotFoundException ex) {
        return Response
            .status(Response.Status.NOT_FOUND)
            .entity(new ApiError(404, "NOT_FOUND",
                ex.getMessage() != null ? ex.getMessage()
                                        : "Resource not found"))
            .type(MediaType.APPLICATION_XML)    // XML error body
            .build();
```

```
    }
}

// Response sent to client:
// HTTP 404 Not Found
// Content-Type: application/xml
// <error>
//   <status>404</status>
//   <code>NOT_FOUND</code>
//   <message>Product not found: P999</message>
// </error>
```

4.5 ContainerRequestFilter — Authentication

```
import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerRequestFilter;
import javax.ws.rs.core.*;
import javax.ws.rs.ext.Provider;

@Provider
@Priority(Priorities.AUTHENTICATION)
public class BearerTokenFilter implements ContainerRequestFilter {

    @Override
    public void filter(ContainerRequestContext ctx) {
        String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
        if (auth == null || !auth.startsWith("Bearer ")) {
            ctx.abortWith(Response
                .status(Response.Status.UNAUTHORIZED)
                .entity(new ApiError(401, "UNAUTHORIZED",
                    "Missing or invalid Bearer token"))
                .type(MediaType.APPLICATION_XML)  // XML error
                .build());
        }
        // else: validate token, set SecurityContext, etc.
    }
}
```

4.6 ContainerResponseFilter — CORS Headers

```
import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.*;
import javax.ws.rs.ext.Provider;

@Provider
@Priority(Priorities.HEADER_DECORATOR)
public class CorsFilter implements ContainerResponseFilter {

    @Override
    public void filter(ContainerRequestContext  req,
                       ContainerResponseContext resp) {
        MultivaluedMap<String,Object> h = resp.getHeaders();
        h.add("Access-Control-Allow-Origin",  "*");
        h.add("Access-Control-Allow-Methods",
              "GET, POST, PUT, DELETE, OPTIONS");
```

```
        h.add("Access-Control-Allow-Headers",
              "Authorization, Content-Type");
        h.add("Access-Control-Max-Age",       "86400");
    }
}
```

4.7 Provider Priority

Priority Constant	Value	Use
Priorities.AUTHENTICATION	1000	Authentication and identity verification
Priorities.AUTHORIZATION	2000	Access control checks
Priorities.HEADER_DECORATOR	3000	Modifying headers (CORS, X-headers)
Priorities.ENTITY_CODER	4000	Compression / decompression
Priorities.USER	5000	Default for user-defined filters

Request filters execute in ascending priority order (1000 first). Response filters execute in descending order (5000 first). This ensures authentication runs before business logic on the way in, and after it on the way out.

4.8 Name Binding — Selective Filter Application

```
// 1. Define the binding annotation
import javax.ws.rs.NameBinding;
import java.lang.annotation.*;

@NameBinding
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface Secured {}

// 2. Annotate the filter with the binding
@Provider
@Secured
@Priority(Priorities.AUTHENTICATION)
public class SecuredFilter implements ContainerRequestFilter { ... }

// 3. Apply to a class (all methods secured)
@Path("/admin")
@Secured
public class AdminResource { ... }

// 4. Apply to a single method only
@Path("/products")
public class ProductResource {
    @GET
    public List<Product> listAll() { ... }  // public — no filter

    @POST
    @Secured                                // only POST requires auth
    public Response create(Product p) { ... }
}
```

4.9 Review Questions

11. Write an ExceptionMapper that catches IllegalArgumentException and returns 400 Bad Request with an XML <error> body.
12. Explain the execution order of two ContainerRequestFilters with priorities 1000 and 3000.
13. When would you use @NameBinding over a global filter? Give a concrete example.
14. What JAXB annotation must the error response bean carry for Jersey to serialise it to APPLICATION_XML automatically?

5. Scanning and @ApplicationPath

5.1 Learning Objectives

- Understand how JAX-RS discovers resource and provider classes via classpath scanning
- Control scanning behaviour with getClasses(), getSingletons(), and servlet parameters
- Understand @ApplicationPath and its interaction with web.xml URL patterns
- Configure multiple JAX-RS applications in a single WAR
- Apply best practices for production deployments

5.2 Automatic Scanning

When the Application subclass returns empty sets from both getClasses() and getSingletons(), the JAX-RS runtime performs classpath scanning to discover all classes annotated with @Path or @Provider within the WAR. This is triggered by the Servlet 3 SPI via the ServletContainerInitializer mechanism.

> **How Classpath Scanning Works**
> 1. Tomcat starts and finds Jersey's ServletContainerInitializer.
> 2. The initialiser scans all .class files in WEB-INF/classes and WEB-INF/lib/*.jar.
> 3. It registers every class annotated with @Path or @Provider.
> 4. It creates a Servlet bound to the @ApplicationPath URI.
>
> If EITHER getClasses() or getSingletons() returns a non-empty set,
> scanning is BYPASSED. You must register EVERY component manually.

5.3 Multiple Applications in One WAR

```
// Version 1 of the API — produces XML
@ApplicationPath("/api/v1")
public class ApiV1Application extends Application {
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(
            V1ProductResource.class,
            V1OrderResource.class
        );
    }
}

// Version 2 of the API — also produces XML
@ApplicationPath("/api/v2")
public class ApiV2Application extends Application {
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(
            V2ProductResource.class,
            V2OrderResource.class
        );
    }
}

// Admin API — restricted, returns XML error bodies
@ApplicationPath("/admin")
```

```
public class AdminApplication extends Application {
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(AdminResource.class, AuditResource.class);
    }
}
```

> **Multiple Application Rules**
> Each Application must have a UNIQUE @ApplicationPath value.
> Providers registered in one Application are NOT visible in another.
> Shared providers (e.g. CorsFilter, NotFoundExceptionMapper)
> must be registered in every Application that needs them.

5.4 Complete Production Application Class

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;
import javax.xml.bind.JAXBContext;
import java.util.Set;

@ApplicationPath("/api")
public class ShopApplication extends Application {

    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(
            // Resources
            ProductResource.class,
            OrderResource.class,
            CustomerResource.class,
            // Filters
            BearerTokenFilter.class,
            CorsFilter.class,
            RequestLoggingFilter.class,
            // Exception mappers — return XML error bodies
            NotFoundExceptionMapper.class,
            ValidationExceptionMapper.class,
            GlobalExceptionMapper.class
        );
    }

    @Override
    public Set<Object> getSingletons() {
        // Shared JAXBContext — expensive to create
        return Set.of(new JaxbContextResolver());
    }
}

// JaxbContextResolver — customise the shared JAXBContext
import javax.ws.rs.ext.*;
import javax.xml.bind.JAXBContext;

@Provider
public class JaxbContextResolver
        implements ContextResolver<JAXBContext> {

    private final JAXBContext ctx;

    public JaxbContextResolver() throws Exception {
```

```
        // Register all JAXB-annotated domain classes
        this.ctx = JAXBContext.newInstance(
            Product.class, Order.class,
            Customer.class, ApiError.class);
    }

    @Override
    public JAXBContext getContext(Class<?> type) {
        return ctx;   // Jersey uses this instead of creating its own
    }
}
```

5.5 Scanning vs Manual Registration — Decision Guide

Scenario	Approach	Reason
Small app, single team	Scanning (empty Application)	Zero config; new classes auto-discovered
Microservice, controlled surface	Manual getClasses()	Only expose intended endpoints
Multiple API versions	Multiple Applications + getClasses()	Isolate v1 and v2 resources cleanly
Shared JAXB context	getSingletons() with ContextResolver	Expensive JAXBContext created once
Production performance-critical	Manual registration	No scanning overhead at startup

5.6 Review Questions

15. Under what exact condition does the JAX-RS runtime skip classpath scanning?
16. A WAR is deployed at /shop with @ApplicationPath("/api"). What is the full URL to GET /api/products/P001?
17. You have 50 resource classes and want to exclude exactly 3 from production. What is the cleanest approach?
18. Why should a JAXBContext be registered as a singleton via getSingletons() rather than created inside each resource method?

Appendix — Quick Reference

Annotation Reference

Annotation	Target	Description
@ApplicationPath	Application subclass	Sets base URI for all resources
@Path	Class / Method	URI path template; variable segments {name}
@GET / @POST / etc.	Method	HTTP method binding
@Produces	Class / Method	Response media type(s) — use APPLICATION_XML for XML
@Consumes	Class / Method	Accepted request media type(s) — use APPLICATION_XML
@PathParam	Parameter	Inject URI template variable
@QueryParam	Parameter	Inject query string parameter
@HeaderParam	Parameter	Inject HTTP request header
@DefaultValue	Parameter	Default value when parameter is absent
@Context	Parameter / Field	Inject UriInfo, HttpHeaders, Request etc.
@Provider	Class	Marks a JAX-RS extension class
@NameBinding	Annotation type	Meta-annotation to create binding annotations
@Priority	Class	Execution order for filters; lower = earlier

MediaType Constants — XML-Relevant

Constant	MIME Type	Use
MediaType.APPLICATION_XML	application/xml	Standard XML REST responses (JAXB)
MediaType.TEXT_XML	text/xml	Alternative XML MIME type (SOAP default)
MediaType.TEXT_PLAIN	text/plain	Simple string responses
MediaType.APPLICATION_OCTET_STREAM	application/octet-stream	Binary downloads

HTTP Status Code Summary

Code	Name	When to use
200	OK	Successful GET, PUT, PATCH

201	Created	Successful POST — include Location header
204	No Content	Successful DELETE or PUT with no body
400	Bad Request	Invalid request body or parameters
401	Unauthorized	Missing or invalid authentication token
403	Forbidden	Authenticated but not authorised
404	Not Found	Resource does not exist
406	Not Acceptable	No variant matches client Accept header
415	Unsupported Media Type	Content-Type not supported by endpoint
422	Unprocessable Entity	Request syntactically valid but semantically invalid
500	Internal Server Error	Unhandled server-side exception